Multicore exercises

State fixed or growing work, worker count, overhead, and metric. Speedup compares elapsed times; efficiency divides speedup by workers.

CH07-Q01 Amdahl targets

A fixed workload has 90%parallel baseline time, 10%serial, and no overhead.

1. Find eight-worker speedup/efficiency.
2. Find the infinite-worker bound and distinguish finite attainability.
3. With 3%serial, find 128-worker speedup.
4. Compare eight workers with 5%serial against 16 workers with 10%serial.
5. For 100 workers and target 90 speedup, find minimum parallel fraction and serial percentage.

CH07-Q02 Fixed overhead and imbalance

10000 equal-cost units plus fixed launch 10 use 100 workers. One worker gets 2%of work, the other 99 evenly divide the remainder.

1. Find heavy/other worker work and total elapsed time.
2. Use 10010 baseline for speedup/efficiency.
3. Repeat for 5%heavy work.
4. What does work stealing improve, and which two overhead categories does it add?

CH07-Q03 Strong and weak scaling

1. A fixed problem takes 80 s on four cores versus 300 s on one. Find speedup/efficiency.
2. A four-times-larger problem keeps per-core work fixed and takes 90 s on four cores; the smaller one-core problem takes 75 s. Find weak efficiency. With linear single-core cost, find scaled speedup and state the compared sizes.
3. Ten additions remain serial; 100 or 10000 additions parallelize. At cost t per addition, find times/speedups for 10 and 100 workers.
4. Why does the small case improve 5.5→10 while the large reaches 91?
5. Does lower strong efficiency prove a race? Give at least three alternatives.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Multicore exercises (continued)

CH07-Q04 Address spaces and publication

1. U has one physical address space and uniform latency; M has private physical spaces per node. Classify them.
2. What correctness requirements apply to SMP object publication? Is coherence sufficient?
3. Initially data=0 and ready=0. There is one writer and no other writes to either object. Explain when this acquire synchronizes and why r=42. Does acquiring 0 then reading data establish that relation?

```
data = 42
release store ready = 1
acquire load ready           // repeat until 1
r = data
```

4. In a separate incorrect variant, both data and ready are atomic and all four accesses use relaxed ordering. The writer stores data=42 then ready=1; the reader loads ready then data. Show an allowed ready=1, r=0 observation. Why is atomicity present without cross-object synchronization? If both objects are plain non-atomic with no other synchronization, why can results not be limited to 0 and 42?
5. Why does message passing require serialization/copy, routing, and optional ACK? Does ACK repair an application's unrelated race?

CH07-Q05 Hardware multithreading

A four-wide core often waits 200 cycles on an LLC miss while serving independent threads.

1. Compare fine/coarse-grained threading and SMT.
2. Does fine-grained threading guarantee lower single-thread latency?
3. Why can SMT fill issue slots but lower per-thread cache hit rates?
4. Average issue is 2.2 although single-thread ILP can fill four slots. Where may SMT help, and which downside metrics need measurement?

Multicore exercises (continued)

CH07-Q06 SIMD costs and Amdahl

Case A has 80 vectorizable one-cycle scalar operations and 20 nonvectorizable ten-cycle operations. Eight fully used lanes accelerate only the first group; ignore loads, setup, clock, and power changes.

1. Find each baseline/new time, accelerated baseline-time fraction, and overall speedup.
2. Independent case B has 100 one-cycle operations, 80 vectorizable. Find eight-lane speedup and the speedup and average utilization on eight physical lanes when at most six operations execute per cycle and the final partial batch occupies one cycle. Do not reuse A's time fraction.
3. How does vector length handle tails and varying lane counts compared with fixed-width SIMD?
4. Why need doubled SIMD width not double overall speedup? List excluded costs.

CH07-Q07 Network counts

Use a 4 x 4 endpoint grid. Mesh has adjacent local links; torus adds row/column wraps. Compare K 16 and a 16-port crossbar. Count each full-duplex physical link once; bisection is the minimum balanced-cut edge count.

1. Derive edge counts from rows/columns and count wraps.
2. Find mesh degree distribution, diameter, bisection, and the torus equivalents.
3. Give all eight wrap endpoint pairs and explain why removing any breaks the degree-four torus.
4. Find K 16 links, degree, diameter, bisection, and one-way cut bandwidth for per-link rate b.
5. Is crossbar endpoint degree the same? What grows quadratically?
6. Why does a uniform-traffic bisection summary not predict sparse-matrix performance?

Multicore exercises (continued)

CH07-Q08 Hierarchical Roofline

A multicore has PC=200 GFLOP/s and DRAM 40 GB/s, decimal units.

1. Derive the single-level roof and ridge.
2. Bound A at I=0.4 and B at I=12; identify bottlenecks.
3. Another phase performs $F = 10^9$ FLOPs at $P_C = 1000$ GFLOP/s. DRAM moves $Q_D = 0.25 \times 10^9$ bytes at $B_D = 100$ GB/s; shared memory moves $Q_S = 4 \times 10^9$ bytes at $B_S = 1000$ GB/s. Find intensities, memory bounds, and compute time.
4. Find cross-layer performance/time bounds. Explain DRAM(4, 400), shared(0.25, 250), and final(4, 250), including why 250 is not shared's plateau.
5. Is a measured 30 GFLOP/s sufficiently explained by being below the roof? Give at least two diagnostics.

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design, Revised Fourth Edition, Chapter 7. Lin Cing, handwritten computer organization notes; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study